ARCHITECTURE & CODE DOCUMENTATION

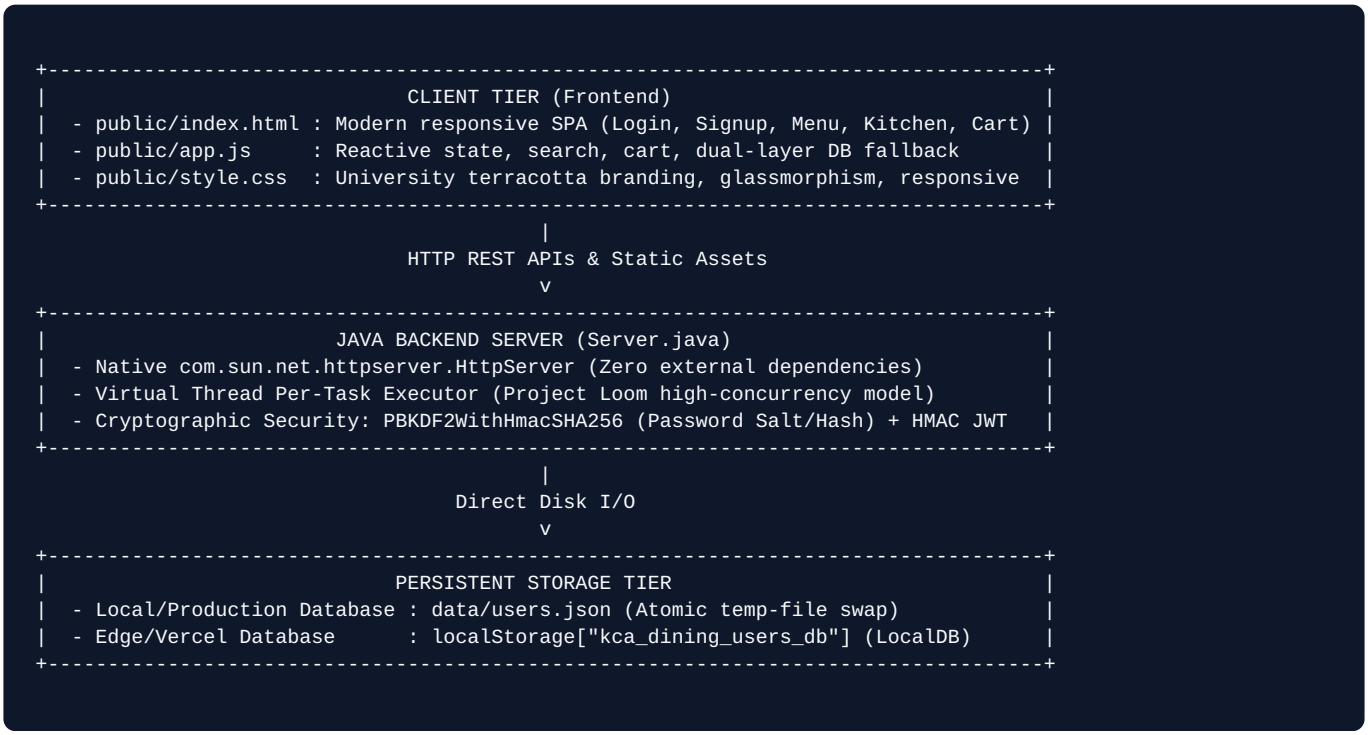
KCAU Smart Dining System

Complete technical walkthrough of backend services, persistent database, frontend reactive state, and deployment architecture.

INSTITUTION	LEAD AUTHOR	BACKEND ENGINE	DATABASE & HOST
KCA University (KCAU)	Solomon (solomon476)	Java 17+ (Virtual Threads)	JSON Engine / Vercel Edge

1. System Architecture Overview

The **KCAU Smart Dining System** is engineered as a decoupled, full-stack campus dining platform. It handles student meal discovery, online ordering, simulated M-Pesa STK push mobile payments, digital receipts, and live kitchen order display queues.



Dual Deployment Model: When executed locally, the app runs on `Server.java` on `http://localhost:3000` with native file persistence. When deployed to **Vercel**, the static bundle uses the resilient `LocalDB` client-side database so accounts and sessions work reliably in both environments.

Native Standalone Java

Runs with a single command `java Server.java` with zero Maven/Gradle build tool dependencies.

Cryptographic Security

10,000-iteration PBKDF2 password hashing with 16-byte random salt and HMAC-SHA256 JWT tokens.

Permanent JSON Database

All registered accounts are safely committed to `data/users.json` and reloaded upon restart.

Kenyan Fintech Simulation

Interactive Safaricom M-Pesa STK push modal with automatic receipt generation and live queueing.

2. Backend Architecture (Server.java)

`Server.java` is a single-file Java backend application utilizing standard Java SE libraries (`com.sun.net.httpserver`, `javax.crypto`, `java.nio.file`).

2.1 Lightweight Virtual-Threaded HTTP Server

```
// Server initialization with Project Loom Virtual Threads
HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
server.start();
```

Rather than dedicating heavy OS threads to every connection, virtual threads handle high-volume concurrent traffic with negligible RAM consumption, ideal for university rush hours.

2.2 Persistent JSON Database Engine (Database Class)

The internal `Database` class coordinates data persistence in `data/users.json`:

- **Automated Initialization (`Database.init()`):** Ensures the `data/` folder exists. If `users.json` is missing, it writes the initial seed student (`KCA/1234/23`). Otherwise, it parses existing users into an in-memory `ConcurrentHashMap`.
- **Safe Atomic Writes (`Database.saveAllUsers()`):** Formats user records into JSON, writes to `data/users.tmp`, and atomically swaps it with `data/users.json` (`StandardCopyOption.ATOMIC_MOVE`). This guarantees zero file corruption even during abrupt server shutdowns.
- **Case-Insensitive Normalization:** Student IDs are normalized using `studentId.trim().toUpperCase()` so that `kca/1234/23` and `KCA/1234/23` match seamlessly.

2.3 Cryptography & Token Engine

Component	Standard	Technical Specification
Password Hash	`PBKDF2WithHmacSHA256`	16-byte random salt, 10,000 iterations, 256-bit key length. Stored format: `Base64(salt):Base64(hash)`.
Token Auth	`HMAC-SHA256 JWT`	Stateless JWT with Header, Payload (`id`, `name`, `studentId`, `exp`), and signature. Valid for 7 days.
Verification	`MessageDigest.isEqual()`	Constant-time byte comparison prevents side-channel timing attacks during authentication.

2.4 Backend REST API Endpoints

Method	Endpoint	Request Body	Description & Status Code
`POST`	`/api/signup`	`{ name, studentId, password }`	Registers account, hashes password, saves to `data/users.json`. Returns `201 Created`.
`POST`	`/api/login`	`{ studentId, password }`	Validates credentials, issues signed JWT token, returns user details. Returns `200 OK`.
`GET`	`/api/me`	`Bearer <token>` Header	Validates JWT signature & expiry, returns profile for active session. Returns `200 OK`.
`GET`	`/api/status`	*None*	System diagnostics: uptime, DB file status, and registered user count. Returns `200 OK`.
`GET`	`/*`	*None*	Static file server for `public/` with SPA fallback to `index.html`.

3. Frontend Client Architecture (public/)

The client interface is built with vanilla web technologies (HTML5, CSS3, ES6+ JavaScript), ensuring blazing speed, accessibility, and zero bundling complexity.

public/index.html (Single-Page View Controller)

Manages Sign In, Registration, Cafeteria Menu, Kitchen Display, and Checkout Modals via display toggles without page refreshes.

public/app.js (Reactive Controller)

Handles live dish filtering, shopping cart state, M-Pesa STK push simulation, toast alerts, and dual-layer database fallback.

public/style.css (Design System)

CSS Custom Properties tokens, modern glassmorphism, responsive grid/flexbox layouts, and KCAU burnt orange terracotta branding.

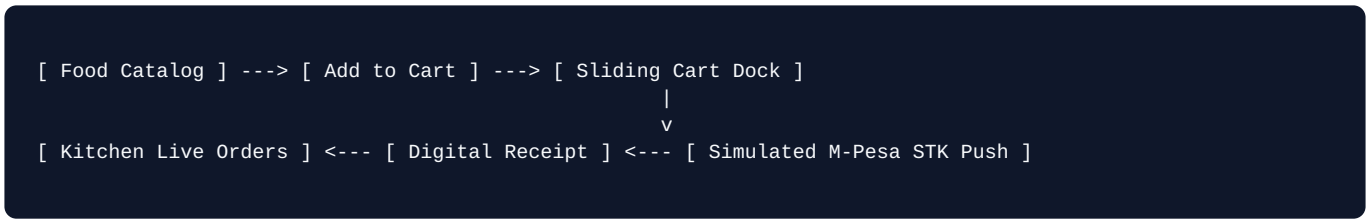
LocalDB (Storage Resilience)

Client-side persistence layer in localStorage ensuring account registration and sign-in work even on static hosts like Vercel.

3.1 Authentication & Sign Out Lifecycle

- **Account Registration:** When a user submits the Sign Up form, `app.js` calls `/api/signup`. Upon creation, it syncs with `LocalDB`, clears input fields, transitions to the Sign In card, and automatically pre-fills the new Student ID.
- **Sign In & Personalization:** Successful authentication stores the JWT in `localStorage.getItem('kca_token')` and the user's name in `localStorage.getItem('kca_user_name')`. The header greeting and hero welcome banner instantly update to greet the user by their registered name.
- **Sign Out:** Clicking the 🚪 **Sign Out** button in the header removes authentication tokens, resets user state, hides the cafeteria menu, and redirects to the login card with an in-app confirmation toast.

3.2 Shopping Cart & M-Pesa Simulation Flow



- **Interactive Cart:** Maintains reactive item quantities, subtotal calculations, and animated badge updates.
- **M-Pesa STK Push:** Prompts the student for their phone number (e.g. `0712345678`), displays simulated payment processing, and verifies the simulated pin authorization.
- **Instant Receipt & Kitchen Sync:** Issues a detailed printable receipt with unique transaction codes and pushes the order to the Kitchen Display System.

3.3 Dual-Layer Database Architecture

```
// Resilient Fallback Pattern in app.js
try {
    const res = await fetch('/api/login', { ... });
    if (res.ok) { handleSuccess(await res.json()); return; }
} catch (netErr) {
    // If backend is unavailable or running on static Vercel host, fall back to LocalDB
    const localRes = LocalDB.authenticate(studentId, password);
    if (localRes.success) { handleSuccess(localRes); return; }
}
```

4. Execution, Deployment & File Reference

4.1 Local Full-Stack Execution (Java Backend)

```
# 1. Clone repository
git clone https://github.com/solomon476/KCAU-SMART-DINING-SYSTEM.git
cd KCAU-SMART-DINING-SYSTEM

# 2. Run Java server directly (JDK 17+)
java Server.java
# Or via npm script: npm start

# 3. Access in browser: http://localhost:3000
```

4.2 Cloud Deployment (Vercel Production)

Resource	URL / Location	Status
Official Production Alias	`https://kca-cafeteria.vercel.app`	✅ Active / Live
Deployment Domain	`https://kcau-smart-dining-system-7888vh9cc-solian-wolves.vercel.app`	✅ Production Ready
GitHub Repository	`https://github.com/solomon476/KCAU-SMART-DINING-SYSTEM`	✅ Main Branch Up to Date

4.3 Codebase File Summary

File	Type	Core Responsibilities
`Server.java`	Java	Standalone HTTP server, virtual threads, PBKDF2 hashing, JWT signing, persistent JSON DB manager.
`data/users.json`	JSON	Permanent database storage holding user credentials, hashed passwords, UUIDs, and creation dates.
`public/index.html`	HTML5	Single-page app markup: login card, signup modal, food category pills, cart, kitchen screen, receipts.
`public/app.js`	JavaScript	Client logic, dual-layer LocalDB fallback, cart manager, M-Pesa STK simulation, live menu search.
`public/style.css`	CSS3	Design system tokens, custom properties, glassmorphism, responsive grid, and university theme styles.
`vercel.json`	JSON	Configuration mapping clean static edge routes to `public/` for global CDN delivery.
`README.md`	Markdown	Project overview, installation guide, REST API documentation, and architecture description.

> **KCAU Smart Dining System** • Engineering university cafeteria operations with speed, security, and elegance.